

Тестовое задание

Junior Frontend Developer / Vue 3 · Nuxt 3

Уровень:	Junior Frontend Developer
Стек:	Vue 3, Nuxt 3, TypeScript, CSS / SCSS
Формат сдачи:	GitHub-репозиторий + ссылка на превью

Это задание — не тест на знание алгоритмов. Мы смотрим на то, как ты думаешь: как структурируешь код, принимаешь архитектурные решения и пишешь компоненты, с которыми приятно работать другим людям.

Тебе предстоит собрать страницу бронирования столика — реальную фичу из нашего продукта, упрощённую до масштаба тестового задания. Сделай её так, как сделал бы в реальном проекте.

** Не старайся охватить всё. Лучше сдать меньше — но аккуратно.*

1. Задача

Реализуй страницу онлайн-бронирования столика в ресторане. Страница должна включать форму бронирования, экран подтверждения и минимальную адаптивную вёрстку.

Что нужно реализовать

Форма бронирования	Имя гостя, номер телефона, дата, время, количество гостей (1-12). Все поля обязательны. Время — только слоты: 12:00, 13:00 ... 22:00.
Валидация	Телефон — российский формат (+7 или 8, 10 цифр). Дата — не раньше сегодня. Ошибки показываются рядом с полем, не алертом.
Отправка формы	Имитируй отправку через setTimeout (1.5 с). Покажи лоадер на кнопке. После — экран подтверждения с деталями.
Экран подтверждения	Показать имя, дату, время, количество гостей. Кнопка «Забронировать ещё» возвращает к форме.

Адаптивность

Корректный вид на мобильном (375 px) и десктопе (1280 px).

2. Технические требования

Обязательный стек

- Vue 3 (Composition API, script setup синтаксис)
- Nuxt 3 (pages/, composables/, layouts/ — на твоё усмотрение)
- TypeScript — типизированы props, emits, composables, reactive-переменные
- CSS Modules, SCSS или Tailwind CSS

Приветствуется, но не обязательно

- Pinia для управления состоянием формы
- VeeValidate или аналог для управления валидацией
- Базовые анимации через <Transition> или CSS transitions
- Линтер (ESLint) и форматтер (Prettier)

Структура проекта (ориентир)

```
components/  
  BookingForm/  
    BookingForm.vue  
    BookingForm.module.scss  
  ConfirmationScreen/  
    ConfirmationScreen.vue  
composables/  
  useBookingForm.ts    // логика формы и валидации  
types/  
  booking.ts           // интерфейсы и типы  
pages/  
  index.vue
```

Пример типов — types/booking.ts

```
export interface BookingFormData {  
  name: string  
  phone: string  
  date: string  
  time: string  
  guests: number  
}  
  
export type BookingStatus = 'idle' | 'loading' | 'success'
```

Пример composable — composables/useBookingForm.ts

```
import { ref, reactive } from 'vue'  
import type { BookingFormData, BookingStatus } from '~/types/booking'
```

```

export function useBookingForm() {
  const status = ref<BookingStatus>('idle')
  const errors = reactive<Record<string, string>>({})
  const form = reactive<BookingFormData>({
    name: '', phone: '', date: '', time: '', guests: 1,
  })

  async function submit() {
    if (!validate()) return
    status.value = 'loading'
    await new Promise(r => setTimeout(r, 1500))
    status.value = 'success'
  }

  return { form, errors, status, submit }
}

```

** Не нужно угадывать «правильный» способ типизации — важно, что типы есть и осмысленны.*

3. Валидация

Ошибки должны появляться в двух случаях: при попытке отправки незаполненной формы и при потере фокуса (@blur) на поле.

Имя гостя	Обязательное. Минимум 2 символа. Только буквы, пробелы, дефис.
Телефон	Обязательное. Формат: +7XXXXXXXXXX или 8XXXXXXXXXX. Допустимы скобки и дефисы при вводе — нормализуй перед валидацией.
Дата	Обязательная. Не раньше сегодня. Не позднее +90 дней.
Время	Обязательное. Выбор из слотов: 12:00–22:00 с шагом 1 час.
Количество гостей	Обязательное. Целое число от 1 до 12.

Пример функции валидации телефона

```
// composables/useBookingForm.ts
function validatePhone(value: string): string | null {
  const digits = value.replace(/\D/g, '')
  if (digits.length === 11 &&
    (digits[0] === '7' || digits[0] === '8')) {
    return null
  }
  return 'Введите корректный номер: +7 или 8, 10 цифр'
}
```

4. UX и дизайн

Мы не ждём дизайна уровня Dribbble — но ждём аккуратности и консистентности.

Цветовая схема	Фон #FAFAF8, акцент #C8963E, текст #1A1A1A. Любая нейтральная палитра.
Типографика	Системный шрифт или Google Font. Иерархия: заголовок → подзаголовок → лейблы → ошибки.
Ошибки	Красный текст под полем. Поле с ошибкой — красная обводка. Не алерт.
Лoader на кнопке	Текст «Бронирую...» + анимированный индикатор. Кнопка disabled при отправке.
Подтверждение	Чистый экран: иконка галочки, сводка деталей, кнопка сброса.
Переходы	Используй встроенный компонент <Transition> Vue для анимации смены экранов.

5. Сдача задания

Формат

- Создай публичный репозиторий на GitHub.
- Задеплой проект на Vercel или Netlify (бесплатный план).
- Пришли ссылку на репозиторий и ссылку на превью.

README.md — обязательно

- Как запустить проект локально (команды)
- Какие решения ты принял и почему (коротко, 3–5 предложений)
- Что бы ты доделал при наличии ещё времени

** README — часть оценки. Умение объяснить решения так же важно, как сам код.*

Чего делать не нужно

- Подключать бэкенд или базу данных
- Реализовывать авторизацию или роутинг по страницам
- Идеально полировать дизайн до pixel-perfect
- Покрывать код тестами (будет плюсом, но не обязательно)

6. Критерии оценки

Итоговая оценка — не сумма галочек. Мы смотрим на общее впечатление: насколько с кодом приятно работать и насколько ты думал о следующем разработчике, который откроет этот проект.

Критерий	Вес	На что смотрим
Качество кода	35%	Читаемость, декомпозиция компонентов, отсутствие дублирования, осмысленные имена
TypeScript	20%	Типизированы props, emits, composables; нет необоснованных any; интерфейсы вынесены
Логика и валидация	20%	Все правила работают корректно; edge cases обработаны; форма не отправляется с ошибками
UX и адаптивность	15%	Корректное отображение на 375 px и 1280 px; лоадер, ошибки и подтверждение работают
README	10%	Понятное описание, объяснены архитектурные выборы, упомянуто что осталось бы доделать

Бонусные баллы

- Pinia с корректным defineStore и использованием в компонентах
- VeeValidate или аналог для управления валидацией
- Анимация смены экранов через <Transition> с CSS-классами enter/leave
- Unit-тесты composable useBookingForm (Vitest)